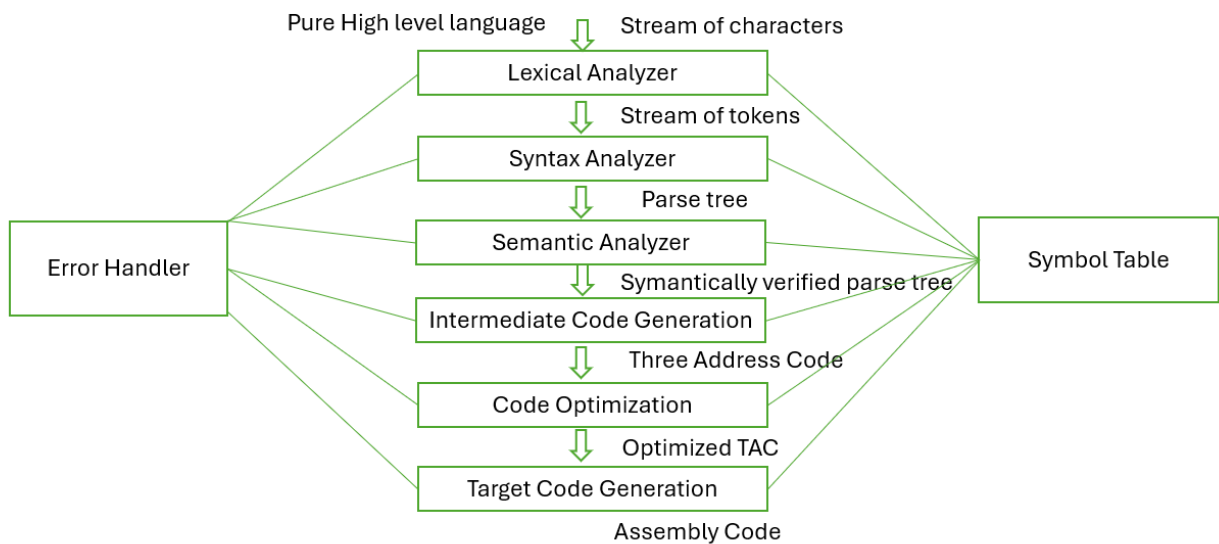


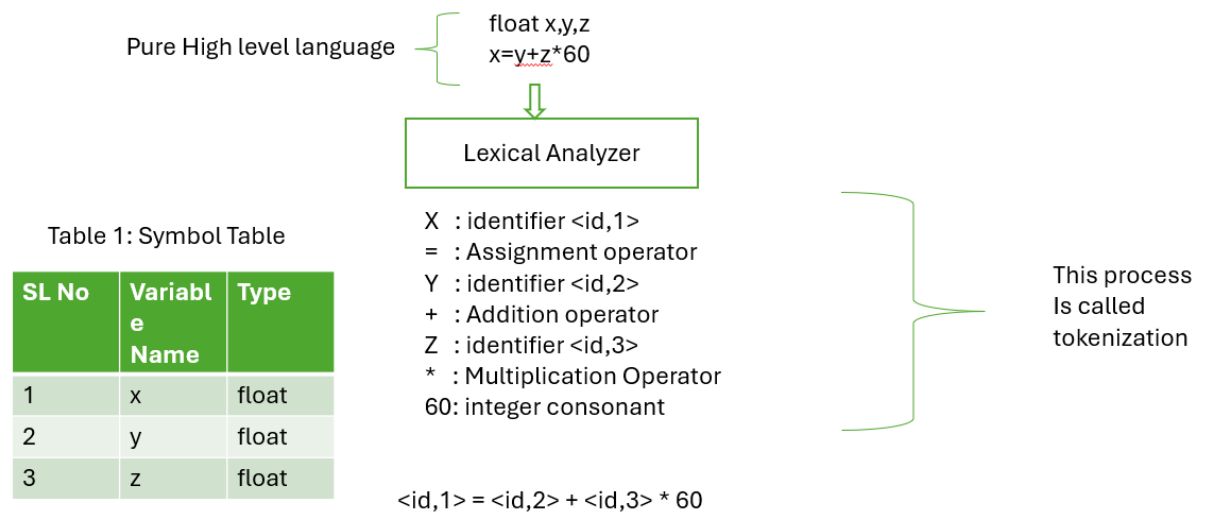
Compiler vs Interpreter

Compiler	Interpreter
Compiler reads the full program.	Interpreter reads line by line.
It generates .exe file.	It doesn't generate .exe file.
Compiled codes run faster than Interpreter. So, it required less time.	Interpreted codes run slower than Compiler. So, it required more time
It does not require source code for later execution.	It requires source code for later execution.
It takes more space.	It takes less space.
Debugging is difficult here.	Debugging is easy here.

6 Phases of Compiler



Lexical Analyzer



Syntax Analyzer

$\langle id, 1 \rangle = \langle id, 2 \rangle + \langle id, 3 \rangle * 60$

Syntax Analyzer

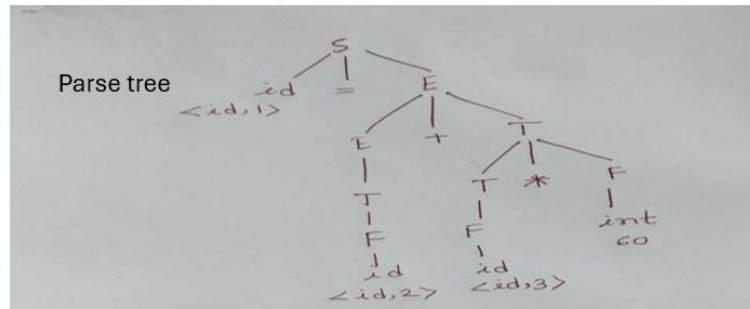
Grammar

$S \rightarrow id = E$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow id \mid int$



Semantic Analyzer

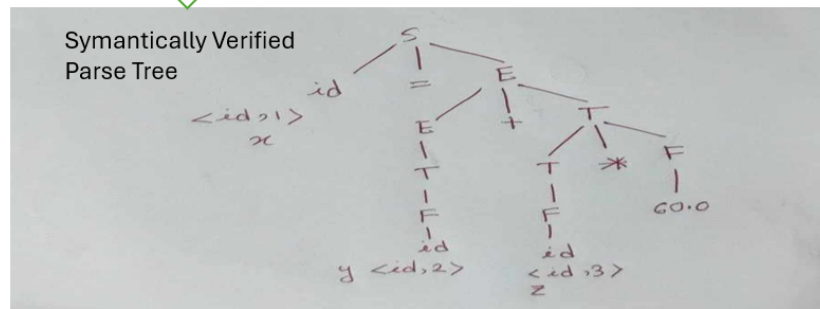
Parse tree

Semantic Analyzer

1. Type Checking
2. Undeclared variable
3. Multiple Declaration

Table 1: Symbol Table

SL No	Variable Name	Type
1	x	float
2	y	float
3	z	float



Intermediate code Generation

- In here the most valuable is three address code.

```
t1 = z*60.0  
t2 = y+t1  
X = t2
```

Code Optimization

- Reduced the no of lines in the code.
- It is optional

```
t1 = z*60.0  
X = y+t1
```

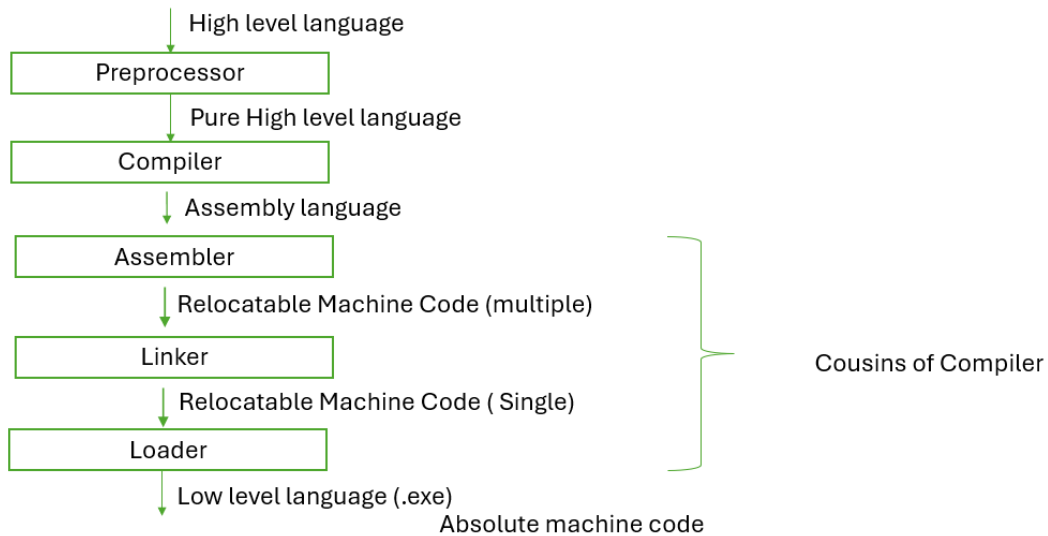
Target Code Generation

- It generates assembly language

```
                                R0 <- z  
                                R1 <- y  
  
    MUL    R0, 60.0  
    ADD    R1, R0  
    STORE  X, R1
```

The **Logic Processing Unit** is essential for enabling computers to make decisions, control signals, and carry out instructions that rely on logical relationships, not just numeric computation.

LPS (Language Processing System)

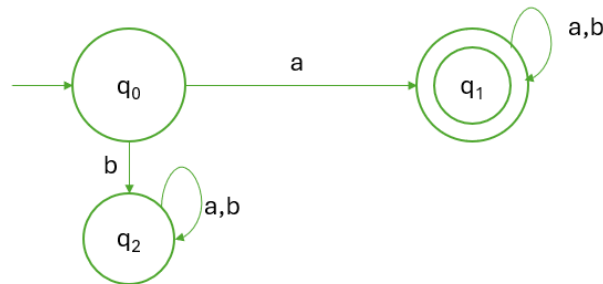


Formal Definition of a DFA

- A DFA can be represented by a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where –
 - Q is a finite set of states.
 - Σ is a finite set of symbols called the alphabet.
 - δ is the transition function where $\delta: Q \times \Sigma \rightarrow Q$
 - q_0 is the initial state from where any input is processed ($q_0 \in Q$).
 - F is a set of final state/states of Q ($F \subseteq Q$).

Example

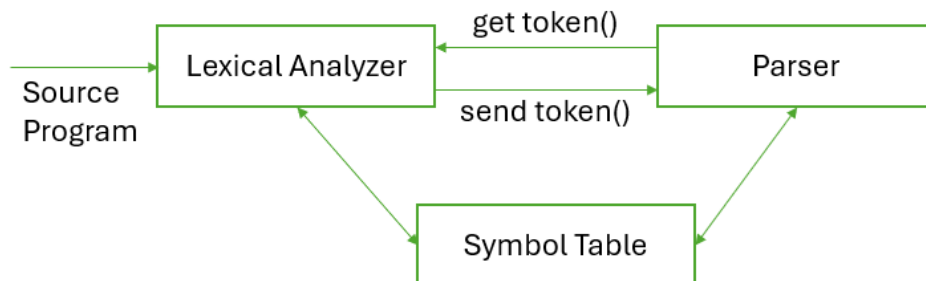
- **Make a DFA where all strings start with a.**
- You have two alphabets (a,b)
 - { a, aa, aaa, ab, ba }



Lexical Analyzer

- Lexical analyzer divides the given program into meaningful words which is known as tokens.
- Tokens are normally identifiers, separator, keywords, operators, constant and special symbol.
- The program which did this is called **lexer, tokenizer, scanner**.
- Lexical analyzer eliminates comments, white space character. (blank, Tab)
- It helps in giving error message such as exceeding length by providing row no and column no.

Lexical Analyzer



Lexical Analyzer

- Lexical analyzer uses DFA to do tokenization.
- While doing tokenization lexical analyzer always gives importance to longest matching.

Counting no of tokens (Ex: 6)

```
1  main() //3
2  { //1
3      char *c="string"; //6
4      float b=100.74; //5
5      char d='e'; //5
6      int f=200; //5
7      in t f=200; //6
8      in /* comment */t f=200; //6
9      ch ar d='e'; //6
10     ch/*comment*/ar d='e';//6
11 } //1
```

Total tokens = 50

Counting no of tokens (Ex: 7)

main.c

```
1  main() //3
2  { //1
3      a=b+++---+++=; //11
4      x==y /*abcd**/*abcd*/; //9
5      int **p; //5
6      printf("%d %d",a,b); //9
7  } //1
```

Total tokens = 39

Error Recovery

- **Delete:** Unknown characters are deleted. Also known as panic mode recovery. Example:
 - 'Charr' corrected as 'char' deleting r.
- **Insert:** An extraa or missing charactered is inserted to form a meaningful token. Example:
 - 'cha' corrected as 'char' inserting 'r'
- **Transpose:** Based on certain rules we can transpose two characters. Example:
 - 'Whiel' can be corrected to 'while'

Error Recovery

- **Replace:** Based on replacing one character by another. Example:
 - 'chrr' can be corrected as 'char' by replacing 'r' with 'a'.

Ambiguous Grammar

- A Grammar that makes more than one Leftmost Derivation (or Rightmost Derivation) (or Parse Tree) for the similar string is called Ambiguous Grammar.

Example1 –

- Verify whether the following Grammar is Ambiguous or Not.
 - $E \Rightarrow E+E \mid E^*E \mid id$

Solution:

For string $id + id * id$

Ambiguous Grammar (Ex:2)

Example2 –

- Verify whether the following Grammar is Ambiguous or Not.
 - $S \Rightarrow aS \mid Sa \mid a$

Solution:

For string 'aa'

Ambiguous Grammar (Ex:2)

LMD1

$S \Rightarrow a\underline{S}$

$\Rightarrow aa$

RMD1

$S \Rightarrow a\underline{S}$

$\Rightarrow aa$

LMD2

$S \Rightarrow \underline{S}a$

$\Rightarrow aa$

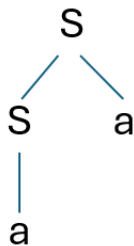
RMD2

$S \Rightarrow \underline{S}a$

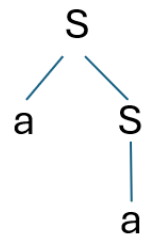
$\Rightarrow aa$

Ambiguous Grammar (Ex:2)

Parse Tree(1)



Parse Tree(2)



Ambiguous Grammar (Ex:2)

- So, the same string is generated using two different leftmost derivations or rightmost derivations.
- Two different parse trees exist for string aa.
- So, Grammar is Ambiguous.

Ambiguous Grammar (Ex:3)

Example3 –

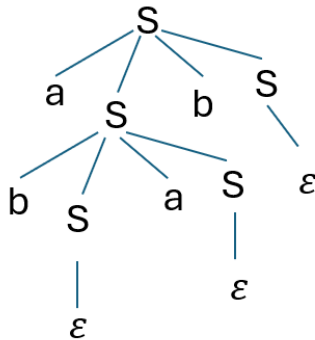
- Verify whether the following Grammar is Ambiguous or Not.
 - $S \Rightarrow aSbS \mid bSaS \mid \epsilon$

Solution:

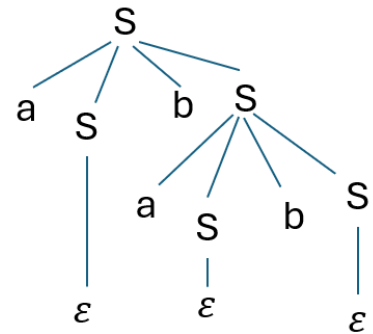
For string 'abab'

Ambiguous Grammar (Ex:3)

Parse Tree(1)



Parse Tree(2)



Ambiguous Grammar

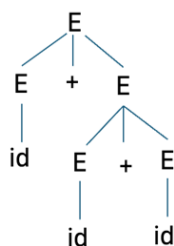
- Two different parse trees exist for string 'abab'.
- So, Grammar is Ambiguous.

Ambiguous to unambiguous grammar

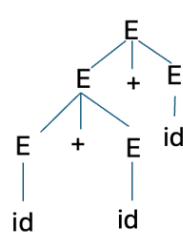
Grammar: $E \Rightarrow E+E \mid E * E \mid id$

$W = id + id + id$

Parse Tree(1)

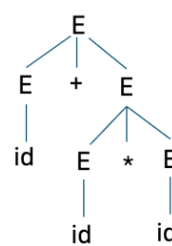


Parse Tree(2)

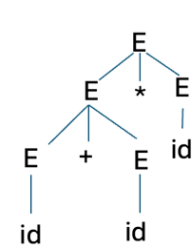


$W = id + id * id$

Parse Tree(1)



Parse Tree(2)

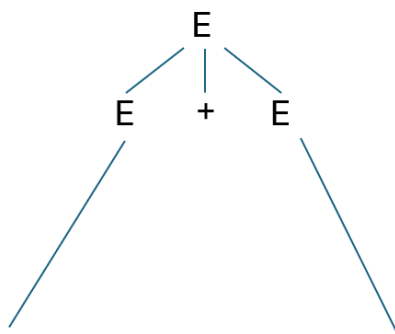


Ambiguous to unambiguous grammar

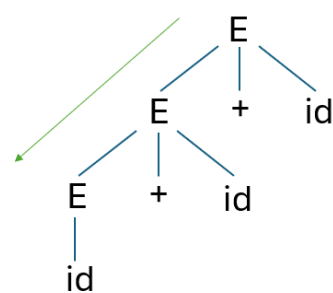
In here we have to give attention to precedence and associativity.

Precedence	Operator	Associativity
3	+, -	Left to right
2	*, /	Left to right
1	↑	Right to left

Ambiguous to unambiguous grammar (Associativity)



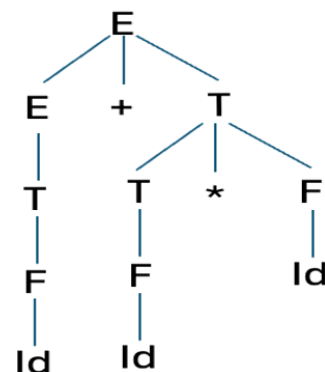
- $E \rightarrow E + id \mid id$



If the grammar is left recursive, then
The operator is left associative

Ambiguous to unambiguous grammar (Precedence)

- When there are different operators, then highest precedence operator should be in the least level.
- Then it will be evaluated first
- So,
- $E \rightarrow E + T \mid T$
- $T \rightarrow T * F \mid F$
- $F \rightarrow id$

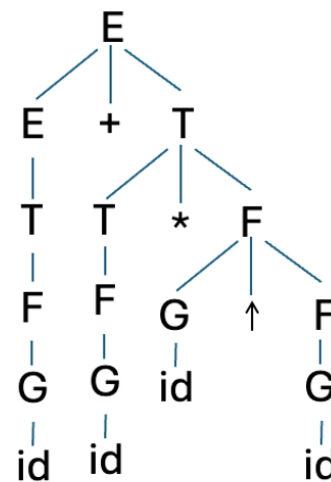


Ambiguous to unambiguous grammar

- In order to define associativity we should define recursion (according to our need i.e. left or right)
- In order to define precedence we should define levels.

Ambiguous to unambiguous grammar

- Let's the string is : $id + id * id^{id}$
- So, the grammar will be:
 - $E \rightarrow E + T \mid T$
 - $T \rightarrow T * F \mid F$
 - $F \rightarrow G \uparrow F \mid G$
 - $G \rightarrow id$



Ambiguous to unambiguous grammar

EX:1

- $R \rightarrow R + R \mid R.R \mid R^* \mid a \mid b \mid c$
- This grammar is ambiguous. Make it unambiguous.

Precedence	Operator	Associativity
3	+	Left to right
2	.	Left to right
1	*	Left to right

Ambiguous to unambiguous grammar

- **EX1: Solution**

- $R \rightarrow R + E \mid E$
- $E \rightarrow EF \mid F$
- $F \rightarrow F^* \mid a \mid b \mid c$